Habit Tracking App — Conception Phase

Portfolio Part 1 — Written Concept

Author: Alejandro Diaz Roque
Matr. No.: <YOUR MATRICULATION NUMBER>
Course: DLBDSOOFPP01 — Object-Oriented and Functional Programming with Python
Date: October 11, 2026
Repository: github.com/<user>/<repo>

---

1 Introduction and Motivation

The client requires a Python backend for a habit tracking application that is scheduled to be rolled out later this year. The task focuses on the essential functionality of such an application — defining habits, checking them off, and analysing progress over time — without providing any graphical user interface. The design must combine **object-oriented programming** for the habit domain and **functional programming** for the analytics layer, and it must expose this logic through a clean, usable interface.

2 Problem Definition and Scope

In scope

- Habit definition with a name, description, and periodicity (at least *daily* and *weekly*).
- Checking off a habit task at any point in time.
- Persistence of habits and their completions between sessions.
- Analytics over the stored data (list habits, filter by periodicity, longest streak overall and per habit).
- A command line interface (CLI) for user interaction.
- A REST API exposing the same core logic over HTTP (FastAPI).
- A unit test suite covering the critical components.

Out of scope

- Any graphical user interface or mobile client.
- Authentication and multi-user support.
- Notifications, reminders, or cloud synchronisation.
- Use or modification of existing third-party habit trackers.

3 Concept and Core Idea

Core idea.

Why a layered architecture?

4 Proposed Solution and Architecture

4.1 Domain Model (Object-Oriented)

The central abstraction is the `Habit` class. Each habit carries a `name`, a `description`, a `periodicity` (`"daily"` or `"weekly"`), a `created_at` timestamp, and a collection of `Completion` objects. A `HabitTracker` orchestrates creation, deletion, check-off, and retrieval of habits. A `Storage` component encapsulates persistence, and an `Analytics` module provides read-only, functional access to the data.

4.2 Analytics Layer (Functional Programming)

The analytics module is deliberately implemented in the functional paradigm. Every analytics function is *pure*: it takes the tracker (or a list of habits) as input and returns a new value without mutating its arguments. Streak computation is expressed as a `functools.reduce` over a sorted, deduplicated sequence of completion dates (or ISO weeks for weekly habits).

4.3 Persistence

Habit data is stored in a **SQLite** database accessed through Python's built-in `sqlite3` module. The schema consists of two tables, `habits` and `completions`, linked by a foreign key with `ON DELETE CASCADE`. This choice avoids external dependencies, supports indices and constraints, and matches the brief's recommendation of a database-backed solution over a plain file store.

4.4 User Interaction: CLI and REST API

Two thin adapters sit on top of the shared core:

- A **CLI** built with `Typer`, offering commands such as `create`, `delete`, `check-off`, `list`, and `analyse longest-streak`.
- A **REST API** built with `FastAPI`, exposing the same operations over HTTP with automatic OpenAPI documentation at `/docs`.

Neither adapter contains business logic; both delegate to `HabitTracker` and `Analytics`.

5 Architecture Diagrams

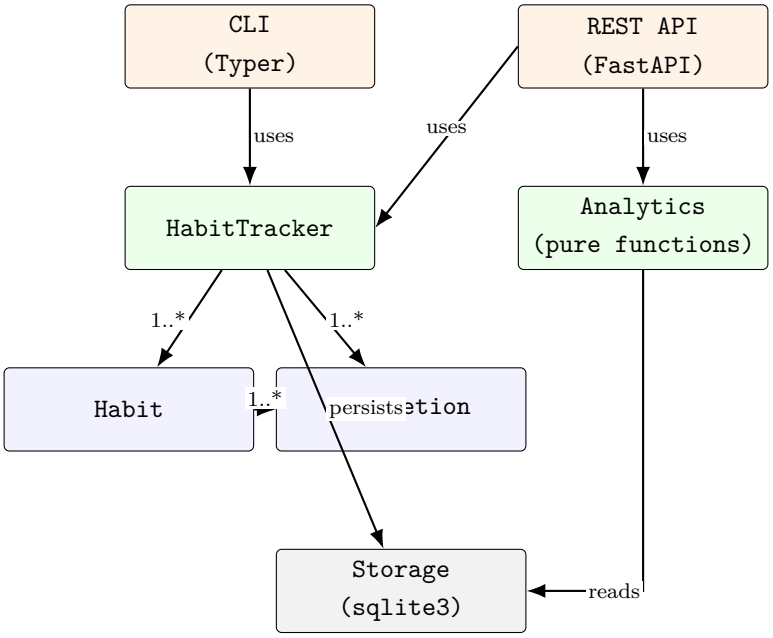


Figure 1: UML class diagram of the habit tracking application. The green nodes form the framework-agnostic core; orange nodes are thin adapters (CLI, REST API); grey is the persistence layer.

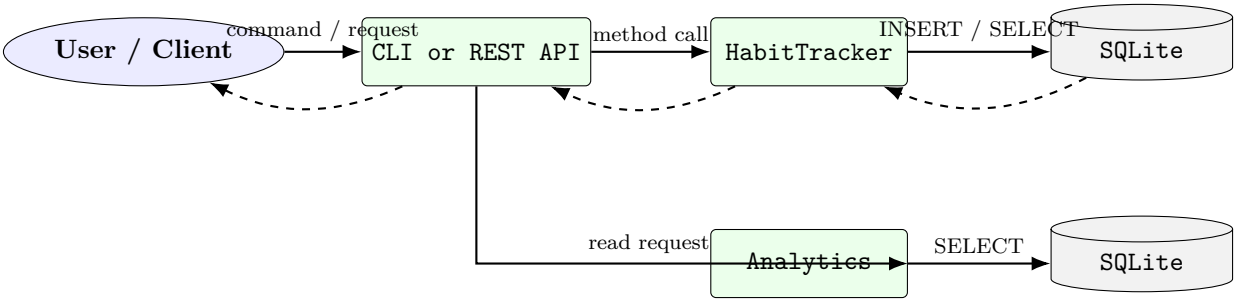


Figure 2: Interaction flow. Top row: write path (`User` → `CLI/API` → `HabitTracker` → `SQLite`). Bottom row: read path for analytics. Both adapters delegate to the same core.

6 Methodology and Tooling

Concern	Tool / Choice	Justification
Language	Python ≥ 3.11	Modern typing, required by the brief (≥ 3.7).
Domain model	OOP (`dataclasses`/classes)	Encodes the habit concept explicitly.
Analytics	Functional paradigm (`map`, `filter`, `reduce`)	Required by the brief for analytics.
Persistence	`sqlite3` (stdlib)	No external deps, real schema, recommended by brief.
CLI	`Typer`	Declarative, testable, auto-help.
REST API	`FastAPI` + `uvicorn`	Automatic OpenAPI docs, Pydantic validation.
Testing	`pytest`, `pytest-cov`, `httpx`	Mature ecosystem, coverage reporting.

Table 1: Overview of chosen tools and the reasoning behind each choice.

7 Plan of Procedure

1. **Phase 1 — Conception (this document).** Define scope, domain model, analytics design, persistence, and interfaces; produce the UML and flow diagrams.
2. **Phase 2 — Development / Reflection.** Implement the core (`Habit`, `HabitTracker`, `Storage`, `Analytics`), then the two adapters (CLI, FastAPI), followed by the fixture data and the unit test suite.
3. **Phase 3 — Finalization.** Refine the implementation after tutor feedback, finalise the README and API documentation, publish the repository on GitHub, and submit the abstract PDF together with a ZIP of all three phases.

8 Testing Strategy

The critical parts of the solution — the validity of the habit tracking components and the analytics module — are covered by a `pytest` suite. Tests target:

- **Models and tracker:** creation, validation errors, check-off, deletion, retrieval, cascade removal of completions.
- **Analytics:** empty history, single completion, consecutive streaks, broken streaks, weekly ISO-week handling, periodicity filtering.
- **Storage:** schema initialisation, round-trip save/load, parameterised queries, foreign-key cascade.
- **Adapters:** CLI smoke tests via `CliRunner`; REST API tests via `TestClient` covering 201/204/404/422 responses.

9 Expected Outcome and Success Criteria

The finished product is a Python package that:

- lets users create, delete, and check off daily and weekly habits;
- persists all state in a local SQLite database;
- ships with five predefined habits and four weeks of example tracking data as a reproducible test fixture;
- provides functional analytics: list all habits, list by periodicity, longest streak overall, longest streak per habit;
- exposes the same logic through both a CLI and a documented REST API;
- includes a passing unit test suite runnable with a single `pytest` command;
- is documented with a README and Python docstrings, and is hosted on a public GitHub repository.

10 Conclusion

The conception phase establishes a clear separation between the framework-agnostic domain core and the two user-facing adapters, aligns each part of the system with the paradigm mandated by the brief (OOP for habits, functional for analytics), and lays out a concrete, testable path from concept to finished product.